EmergencyAI: An AI-Based Emergency Response Coordination and Smart Hospital Resource Allocation System

SAHANA A (2116240701451) and SAHANA P (2116240701451)

Department of Computer Science and Engineering, Rajalakshmi Engineering College, 2026

ABSTRACT

Emergency response depends on timely decisions about how serious a case is, which ambulance to send and which hospital can best receive the patient. When patient information, ambulance availability and hospital readiness are viewed in separate places, comparing options under time pressure becomes difficult. This paper presents EmergencyAI, a web-based educational prototype that brings emergency intake, patient observation analysis, ambulance support and hospital resource recommendations into a single dashboard. The system uses a React and Vite frontend, a Python FastAPI backend with Pydantic validation, SQLAlchemy for data access and a local SQLite database. A rule-based severity engine evaluates patient observations and contextual risk factors, accumulates a score capped at 100 and maps it to one of four categories: LOW, MEDIUM, HIGH or CRITICAL. Allocation logic then ranks possible ambulances and hospitals using availability, estimated distance and travel time, capacity, occupancy, ventilator availability and specialist matching. The prototype runs on demonstration seed data and heuristic rules that have not been clinically or operationally validated, and no performance results are claimed. EmergencyAI is intended to demonstrate the integration of these functions in one application and to provide a base for future, properly validated work.

Keywords: emergency response, ambulance recommendation, hospital resource allocation, rule-based severity scoring, FastAPI, React, decision support

I. INTRODUCTION

In an emergency, the time between a call for help and the arrival of suitable care depends on several decisions made in quick succession. Operators must understand the patient's condition, decide how urgent the case is, find an ambulance that can reach the patient and identify a hospital with the capacity and specialist cover to treat them. Each decision relies on information that may be held in different places, such as call notes, vehicle lists and hospital records.

This paper describes EmergencyAI, a final-year Computer Science and Engineering project that explores how these activities can be combined in one web application. The system lets an operator record an emergency, applies a rule-based severity analysis to the entered observations, and shows suggested ambulances and hospitals alongside the other emergencies, vehicles and facilities on a dashboard.

EmergencyAI is deliberately positioned as an educational prototype. It does not replace professional triage, it is not connected to real emergency services or hospital systems, and its rules are heuristics rather than clinically validated criteria. The contribution of the work is the design and implementation of an integrated prototype, not a proven clinical or operational improvement.

The main contributions are:

- A structured emergency case recording workflow backed by request validation.
- A rule-based prototype severity engine that produces a score capped at 100 and a category.
- Heuristic ambulance and hospital recommendation logic based on available prototype data.
- A unified dashboard with alerts, patient records, analytics and an emergency simulation.

The rest of the paper is organised as follows. Section II gives background. Section III states the problem and objectives. Section IV presents the architecture. Section V describes the methodology. Section VI covers implementation. Section VII discusses what the prototype demonstrates. Sections VIII to X cover limitations, future work and conclusions.

II. BACKGROUND AND RELATED WORK

This section gives conceptual background only. It is not a systematic literature survey.

Early warning and triage scoring. Clinical practice already uses structured, rule-based scoring of vital signs to flag deteriorating patients. Examples include the Modified Early Warning Score [1] and the National Early Warning Score 2 [2], and the Glasgow Coma Scale [3] for level of consciousness. These tools show that simple, transparent rules over observations such as heart rate, blood pressure, respiratory rate, temperature and consciousness are a recognised approach. EmergencyAI is inspired by the general idea of rule-based scoring, but its rules are not taken from, equivalent to or validated against these instruments.

Resource visibility. Coordinating emergency care also involves knowing which ambulances are available and which hospitals have capacity. In some settings this information is spread across different tools or obtained by phone, while other systems are already well integrated. EmergencyAI explores a unified view rather than claiming that existing systems are ineffective.

Web application technologies. The prototype uses widely available open-source tools. FastAPI provides a Python framework for REST interfaces [4], and Pydantic supplies data validation based on type declarations [5]. SQLAlchemy provides database access from Python [6], and SQLite offers a lightweight local database [7]. On the client side, React is used to build the interface [8], and Leaflet supports map visualization [9]. The REST architectural style [10] underlies communication between the frontend and backend.

III. PROBLEM STATEMENT AND OBJECTIVES

A. Problem statement

The prototype addresses four related difficulties:

- **Fragmented patient information:** details may be held in separate places, which slows the assembly of a complete picture.
- **Limited ambulance visibility:** it can be hard to see which units are available and how far away they are.
- **Difficulty comparing hospitals:** capacity, occupancy and specialist cover vary and are slow to compare.
- **Time-pressured review:** operators may need to review several emergencies at once and decide quickly.

B. Objectives

The objectives of EmergencyAI are to:

1. Record emergency cases in a structured format.
2. Provide dashboard monitoring of emergencies, ambulances and hospitals.
3. Calculate a rule-based prototype severity score.
4. Recommend an ambulance for a case.
5. Match a case to hospital resources.
6. Support alerts, patient records, analytics and emergency simulation.

IV. SYSTEM ARCHITECTURE

EmergencyAI follows a layered client-server design. The operator uses a React and Vite frontend, which sends REST API requests to a FastAPI backend. The backend performs request validation and business logic, which in turn uses two components: the rule-based severity engine and the resource allocation logic. Data is read and written through SQLAlchemy to a SQLite database.

Layer	Component	Role
Presentation	React, Vite, React Router, Recharts, Leaflet	User interface, navigation, charts and map-related visualization where configured
Application	Python, FastAPI, Uvicorn, Pydantic	REST services, request validation, business logic
Decision logic	Severity engine, allocation logic	Scoring, categorisation and ranking
Data	SQLAlchemy, SQLite	Data access and local persistence

Table I. Architectural layers and technologies.

The main data entities are Emergency, Patient, Ambulance, Hospital, Alert, HospitalResource and User. Emergencies and patients record the case. Ambulances and hospitals describe the resources, with hospital capacity information held as hospital resources. Alerts support dashboard notification, and users represent system operators.

V. METHODOLOGY

A. System workflow

Processing of an emergency follows eight steps:

1. The operator enters the emergency information.
2. The backend validates the request.
3. Patient observations and contextual risk factors are evaluated.
4. A prototype severity score is calculated.
5. A severity category is assigned: LOW, MEDIUM, HIGH or CRITICAL.
6. Ambulance availability and relevant travel estimates are evaluated.
7. Hospitals are ranked using capacity, occupancy, specialist capability and other available resource information.
8. Recommendations are displayed and records are stored or retrieved.

B. Rule-based severity analysis

The severity engine evaluates a set of indicators that are commonly associated with greater urgency. Examples considered are low oxygen saturation (SpO_2), elevated heart rate, low systolic blood pressure, rapid breathing, elevated temperature, severe injury information, chest pain or breathing difficulty, and loss of consciousness.

Conceptually, each applicable rule contributes to a cumulative score. The score is capped at 100 and then mapped to a severity category, which in turn informs a suggested action. The process is:

Input observations → Rule evaluation → Cumulative score (capped at 100) → Severity category → Suggested action

This paper intentionally does not state specific rule weights or category thresholds. They are prototype heuristics chosen for demonstration and have not been clinically validated, so they must not be used for real patient care decisions.

C. Ambulance recommendation

Ambulance suggestions consider four factors: availability, estimated distance, likely travel time and the urgency of the case. Only units that are available are candidates, and nearer units with shorter estimated travel times are preferred, with urgency influencing how strongly travel time matters.

D. Hospital recommendation

Hospitals are ranked using ICU and emergency capacity, ventilator availability, specialist matching, occupancy, and estimated distance and travel time. The output is a ranked list rather than a single decision, so that the operator can compare options.

Both recommendation steps use prototype data and heuristic ranking. They do not use live operational feeds, live GPS or live traffic information.

VI. IMPLEMENTATION

The prototype is implemented as a local web application. The backend is written in Python using FastAPI and is run with Uvicorn. Pydantic models validate incoming requests and describe data shapes, and SQLAlchemy maps application data to a SQLite database. FastAPI also generates interactive API documentation through Swagger.

The frontend is built with React and Vite. React Router handles navigation between pages, Recharts draws analytics charts, and Leaflet is used for map-related visualization where configured.

In a typical local setup, the backend API runs at http://127.0.0.1:8000, the Swagger documentation is available at http://127.0.0.1:8000/docs, and the frontend usually runs at http://localhost:5173. The database is a local SQLite store populated with demonstration seed data.

The application is organised into seven functional modules:

No.	Module	Purpose
1	Emergency Intake and Case Management	Record structured emergency and patient details
2	Rule-Based Severity Analysis	Score observations and assign a category
3	Ambulance Operations	Track availability and suggest a unit
4	Hospital Operations and Resource Matching	Compare capacity, specialists and distance
5	Dashboard and Alerts	Monitor cases, units and hospitals and view alerts
6	Patient Records and Analytics	Review stored records with charts
7	Emergency Simulation	Run demonstration emergencies to test the workflow

Table II. Functional modules of EmergencyAI.

The planned demonstration sequence is: dashboard, emergency intake, severity output, ambulance recommendation, hospital recommendation, and finally alerts and analytics.

VII. RESULTS AND DISCUSSION

No quantitative experiments have been conducted, and this paper reports no accuracy, speed or clinical outcome figures. The results are therefore functional: the prototype demonstrates that emergency intake, rule-based severity analysis, ambulance recommendation and hospital resource matching can be combined in one web application, with a dashboard, alerts, patient records, analytics and an emergency simulation.

The design offers several practical benefits for a demonstration system. A single interface allows side-by-side review of resources. Structured intake produces consistent records. The modular separation between interface, validation, decision logic and storage makes the system easy to extend, for example by changing the rule set or replacing the database.

These observations are qualitative and describe the design, not measured performance. Any claim that the recommendations improve response times or patient outcomes would require systematic testing with realistic data and qualified professionals, which is outside the scope of this project.

VIII. LIMITATIONS

The current prototype has the following limitations:

- It uses local SQLite storage and demonstration seed data.
- Location coordinates are hard-coded examples.
- There is no live GPS tracking.

- There is no direct integration with hospital information systems or emergency services.
- The rule-based scoring and allocation logic have not been clinically or operationally validated.

Because of these limits, the outputs of EmergencyAI must be treated as illustrations of the approach and not as guidance for real emergencies.

IX. FUTURE WORK

Possible enhancements include:

- A secure backend with authentication.
- Authorized integrations with hospital and ambulance systems.
- Reliable live location and route information.
- Improved monitoring and data protection.
- Systematic testing and evaluation with qualified professionals.
- Usability and accessibility improvements.

In particular, replacing heuristic rules with criteria reviewed and validated by clinicians, and evaluating the allocation logic against realistic operational data, would be necessary before any real-world use.

X. CONCLUSION

EmergencyAI demonstrates how emergency intake, rule-based severity analysis and resource allocation can be integrated in one web application built with React, FastAPI, SQLAlchemy and SQLite. It gives operators a unified view of emergencies, ambulances and hospitals and produces transparent, heuristic recommendations. It is an educational prototype and not a production clinical system. Its value lies in the integrated design and in providing a foundation for future, validated and securely integrated development.

REFERENCES

[1] C. P. Subbe, M. Kruger, P. Rutherford and L. Gemmel, "Validation of a modified Early Warning Score in medical admissions," QJM, vol. 94, no. 10, pp. 521-526, 2001.

[2] Royal College of Physicians, National Early Warning Score (NEWS) 2: Standardising the assessment of acute-illness severity in the NHS. London: RCP, 2017.

[3] G. Teasdale and B. Jennett, "Assessment of coma and impaired consciousness: a practical scale," The Lancet, vol. 304, no. 7872, pp. 81-84, 1974.

[4] FastAPI documentation. [Online]. Available: https://fastapi.tiangolo.com

[5] Pydantic documentation. [Online]. Available: https://docs.pydantic.dev

[6] SQLAlchemy documentation. [Online]. Available: https://docs.sqlalchemy.org

[7] SQLite documentation. [Online]. Available: https://www.sqlite.org/docs.html

[8] React documentation. [Online]. Available: https://react.dev

[9] Leaflet documentation. [Online]. Available: https://leafletjs.com

[10] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Dept. of Information and Computer Science, Univ. of California, Irvine, 2000.